

Sistemas Operacionais

Gerenciamento de Memória

Professor: **Jeremias Moreira Gomes**

E-mail: jeremias.gomes@idp.edu.br

Introdução

Introdução

- Em um “mundo ideal” programadores possuem a disposição
 - Memória grande (o suficiente)
 - Memória muito rápida
 - Memória não volátil
 - Memória de baixo custo
- A tecnologia atual ainda não consegue atender todas essas demandas (infelizmente)

Introdução

- A alternativa para resolver o problema foi aproveitar da melhor forma o conceito de hierarquia de memória
 - O Sistema Operacional de alguma forma aproveita ao mesmo tempo em que abstrai o uso para o usuário
- A parte do Sistema Operacional que gerencia a hierarquia de memórias é chamada de **Gerenciador de Memória**

Gerenciador de Memória

- Gerenciador de memória:
 - Gerencia no nível do próprio sistema
- Sua função é gerenciar eficientemente a memória:
controlar quais partes estão sendo usadas, alocar memória para processos quando eles precisam e liberá-la quando tiverem terminado

Tarefas do Gerenciador de Memória

- Gerenciar a Hierarquia de Memória
 - Gerenciar espaços livres e ocupados
 - Alocar e localizar processos e dados na memória

Tarefas do Gerenciador de Memória

- **Controlar partes de memória em uso**, para:
 - Alocar memória aos processos, por demanda
 - Liberar memória quando um processo termina
 - **Tratar o problema de swapping**
 - Gerenciar o chaveamento entre memória principal e o disco, e memória principal e cache

Gerência de Memória em Monoprogramação

- Inicialmente, não existia a ideia de abstração de memória

```
_start:  
    mov [0x1337], rax  
    nop
```


Gerência de Memória em Monoprogramação

```
_start:  
    mov [0x1337], rax  
    nop
```

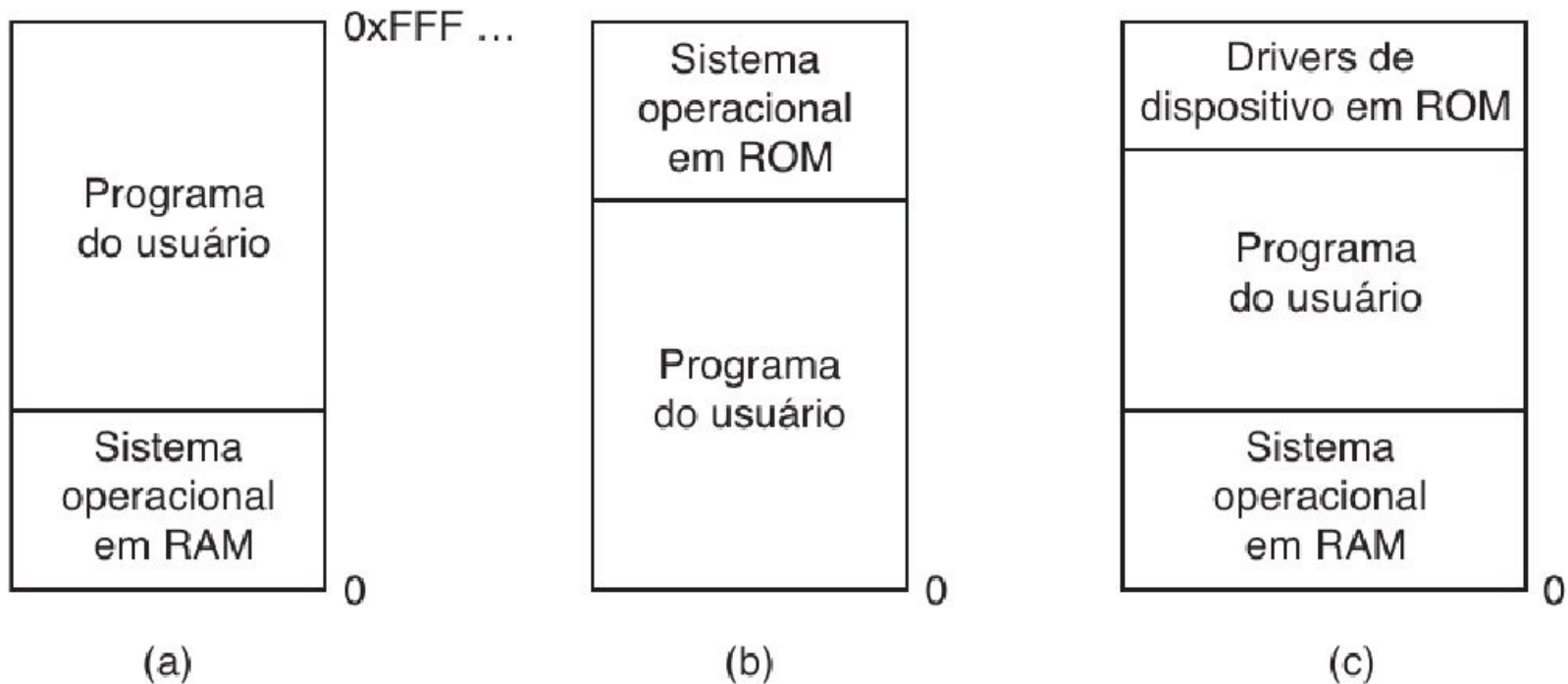
- Cada programa acessava diretamente a memória física
 - Não era possível ter dois programas executando ao mesmo tempo
 - O programa divide a execução com o próprio Sistema Operacional

Gerência de Memória em Monoprogramação

```
_start:  
    mov [0x1337], rax  
    nop
```

- No exemplo, 0x1337 é um endereço físico
- O modelo de memória possibilita algumas organizações simples, do ponto de vista do posicionamento do Sistema Operacional

Gerência de Memória em Monoprogramação



Gerência de Memória em Monoprogramação

- a) O primeiro modelo era utilizado em computadores de grande porte
- b) O segundo foi bastante utilizado em sistemas embarcados
- c) Já o terceiro modelo foi utilizado nos primeiros computadores pessoais
 - ROM era chamada de BIOS (Basic Input Output System)
 - i) Utilizado para realizar a inicialização do hardware durante a inicialização e para fornecer serviços de tempo de execução
 - ii) Hoje foi substituída pela UEFI (Unified Extensible Firmware Interface (UEFI))

Gerência de Memória em Monoprogramação

- Em qualquer uma dessas três organizações, um processo executa por vez
- Sistema Operacional coleta o comando do usuário que copia o programa solicitado do disco para a memória e o executa
 - Quando o programa termina, o SO coleta um novo comando
 - Cada novo programa sobrescreve o anterior
 - Paralelismo só “seria” possível com threads
 - Muito primitivo até para pensar em threads

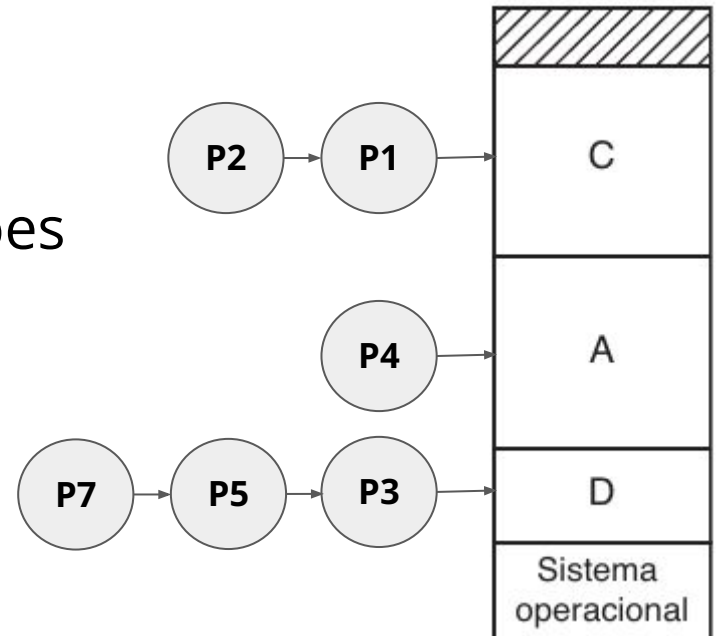
Gerência de Memória em Multiprogramação

- Como armazenar N processos na memória?

- Ideia inicial (1/2):

- Dividir a memória em N partições

- Tamanho fixo



- Cada espaço possui uma fila de execução

Gerência de Memória em Multiprogramação

- Como armazenar N processos na memória?

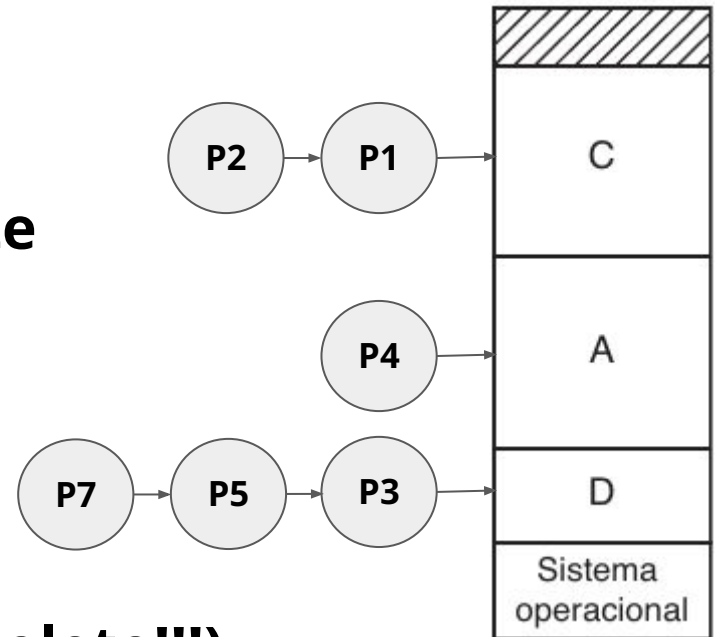
- Ideia inicial (2/2):

- Dois registradores **base** e **limite**

- A CPU aloca base e limite

- Faz-se sempre a verificação

- **Conceito obsoleto (muito obsoleto!!!)**



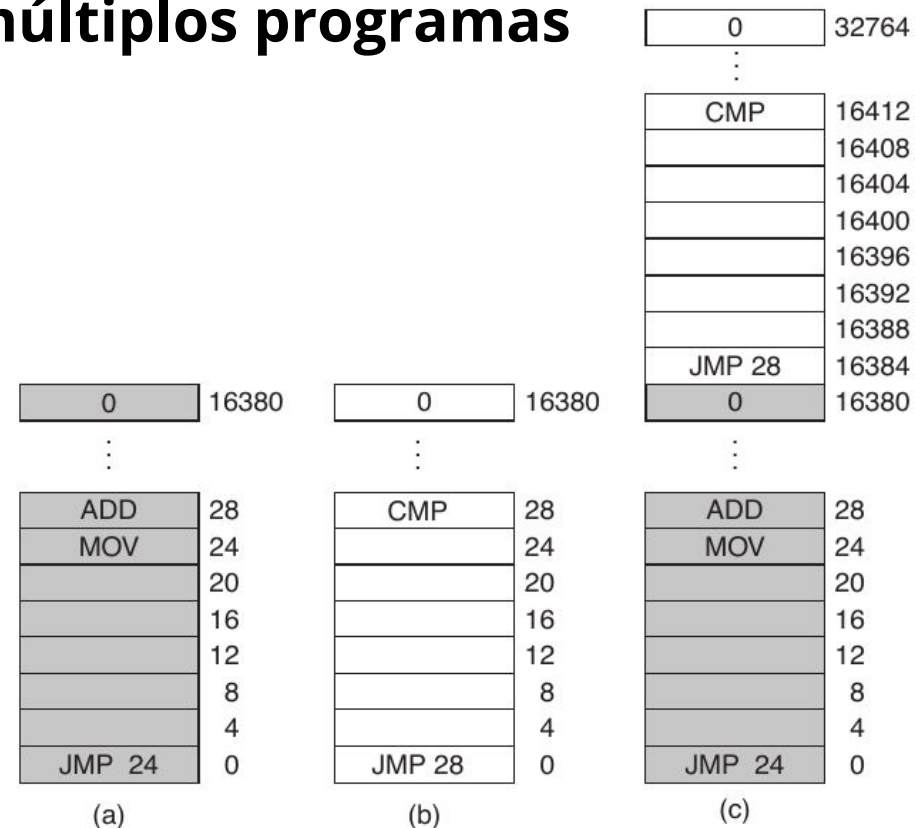
Gerência de Memória em Multiprogramação

- Problema da execução de múltiplos programas

a. Programa 01

b. Programa 02

c. Programas na memória



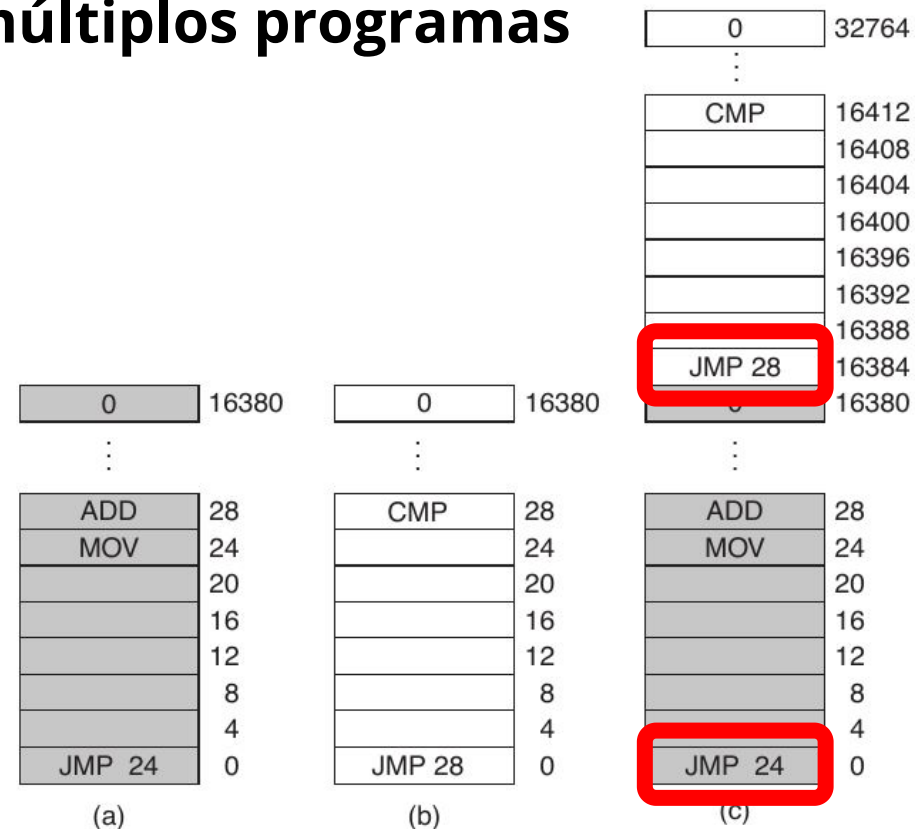
Gerência de Memória em Multiprogramação

- Problema da execução de múltiplos programas

a. Programa 01

b. Programa 02

c. Programas na memória



*Programa b) foi escrito com o um salto para uma região que não é dele, por causa da multiprogramação

Gerência de Memória em Multiprogramação

- Programas devem ter o cuidado de não referenciar memória física absoluta
- Primeiras soluções (IBM 360) usavam **Realocação Estática**
 - Na hora de carregar um programa, um offset era calculado e acrescentado a cada endereço
 - Necessita de informações adicionais sobre endereços

Espaços de Endereçamento

- Primeira ideia sobre abstração de memória
 - **Tentar não referenciar endereços físicos diretamente**
 - Se programas podem referenciar qualquer endereço físico, aumentam-se as chances de derrubar o SO
 - Intencional ou não

Espaços de Endereçamento

- Dois problemas precisam ser resolvidos sobre o acesso a endereços físicos:

1. Proteção

- Não permitir acesso indevido (não pertencente)

2. Realocação

- Posicionar programas em regiões deslocadas

Espaços de Endereçamento

- Solução (para os dois problemas)
 - Inventar uma abstração para a memória
 - Espaço de Endereçamento
- Um **espaço de endereçamento** é o conjunto de endereços que um processo pode usar para endereçar a memória

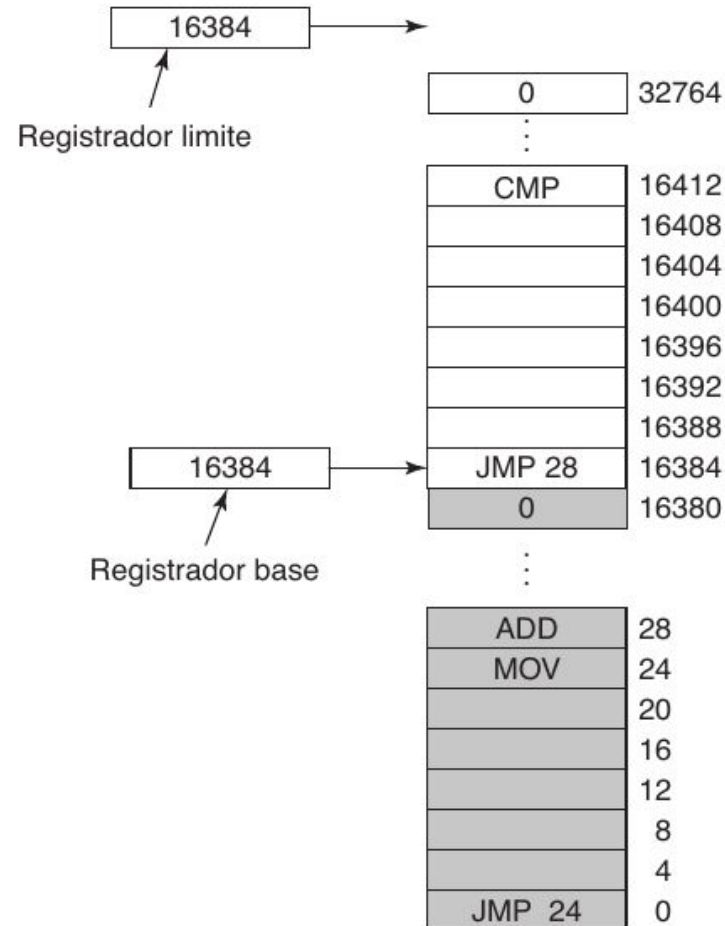
Espaços de Endereçamento

- Da mesma forma como cada processo tem a sua CPU virtual para executar programas, ele agora também tem o seu próprio espaço de endereçamento, independente de outros processos
 - Exceto quando estes querem compartilhar algum espaço

Espaços de Endereçamento

- Aquela ideia obsoleta de registradores base e limite se aplicam à ideia inicial de espaços de endereçamento, onde a cada operação envolvendo memória, o valor do registrador base era adicionado à memória a ser acessada
 - Problema: ter que realizar adições e checagens a cada instrução

Espaços de Endereçamento



Espaços de Endereçamento

- Além das situações citadas, outro problema a ser resolvido no gerenciamento de memória, diz respeito ao tamanho da memória ocupada pelos processos
 - Se a memória física fosse grande o suficiente para todos os processos, até a solução mais simples proposta até agora seria suficiente

Espaços de Endereçamento - MMU

- Hoje em dia, quando um programa referencia um endereço de memória, esse endereço é chamado de endereço virtual
 - Esses endereços formam o Espaço de Endereçamento Virtual

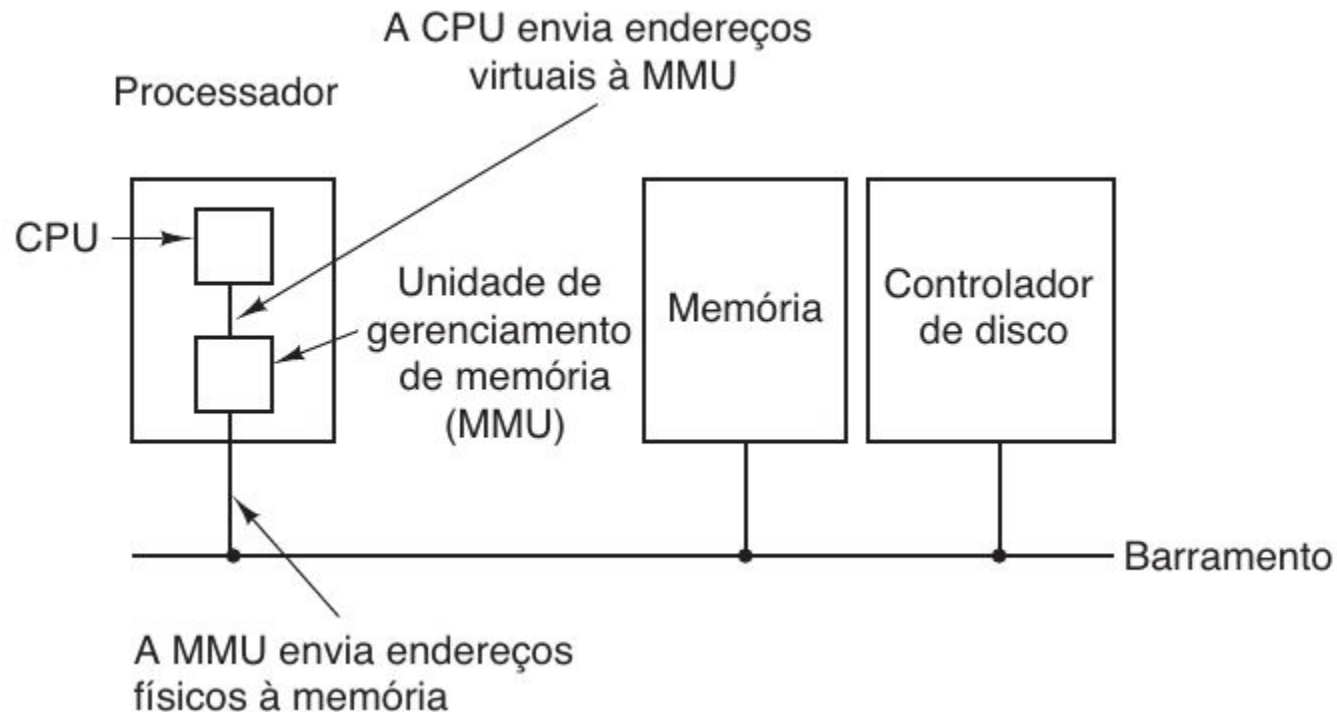
Espaços de Endereçamento - MMU

- Quando Endereçamento Virtual existe, usos de endereços não vão diretamente para o barramento de memória
 - Passam primeiro por um módulo chamado de **Memory**

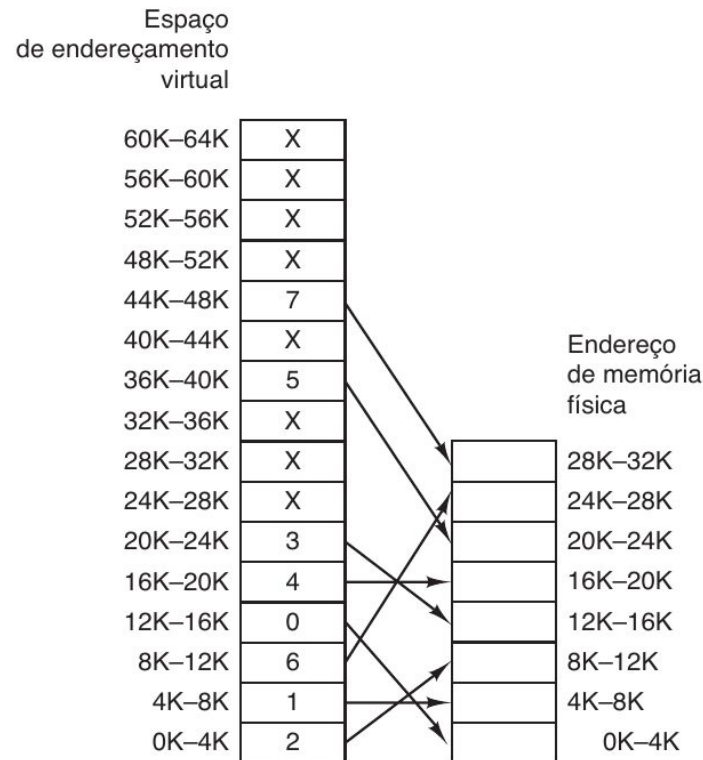
Management Unit (MMU)

- Responsável por mapear endereços virtuais em endereços de memória física

Espaços de Endereçamento - MMU



Espaços de Endereçamento - MMU



Espaços de Endereçamento - MMU

- Com a presença da MMU, um programa nunca tem acesso direto a endereços físicos reais da memória
 - Há um passo intermediário de hardware para a tradução

Espaços de Endereçamento

- Além do que já foi falado, na prática, a quantidade de memória utilizada pelos processos é muito maior do que a memória disponível
- Para resolver esses problemas de sobrecarga, duas abordagens surgiram:
 - **Swapping e Memória Virtual**

Espaços de Endereçamento

- **Swapping**
 - Estratégia “mais simples”
 - Consiste em trazer o processo para a memória em sua totalidade, executá-lo para a sua parcela de tempo e, então, armazená-lo de volta no disco
 - Processos ociosos não ocupam memória

Swapping

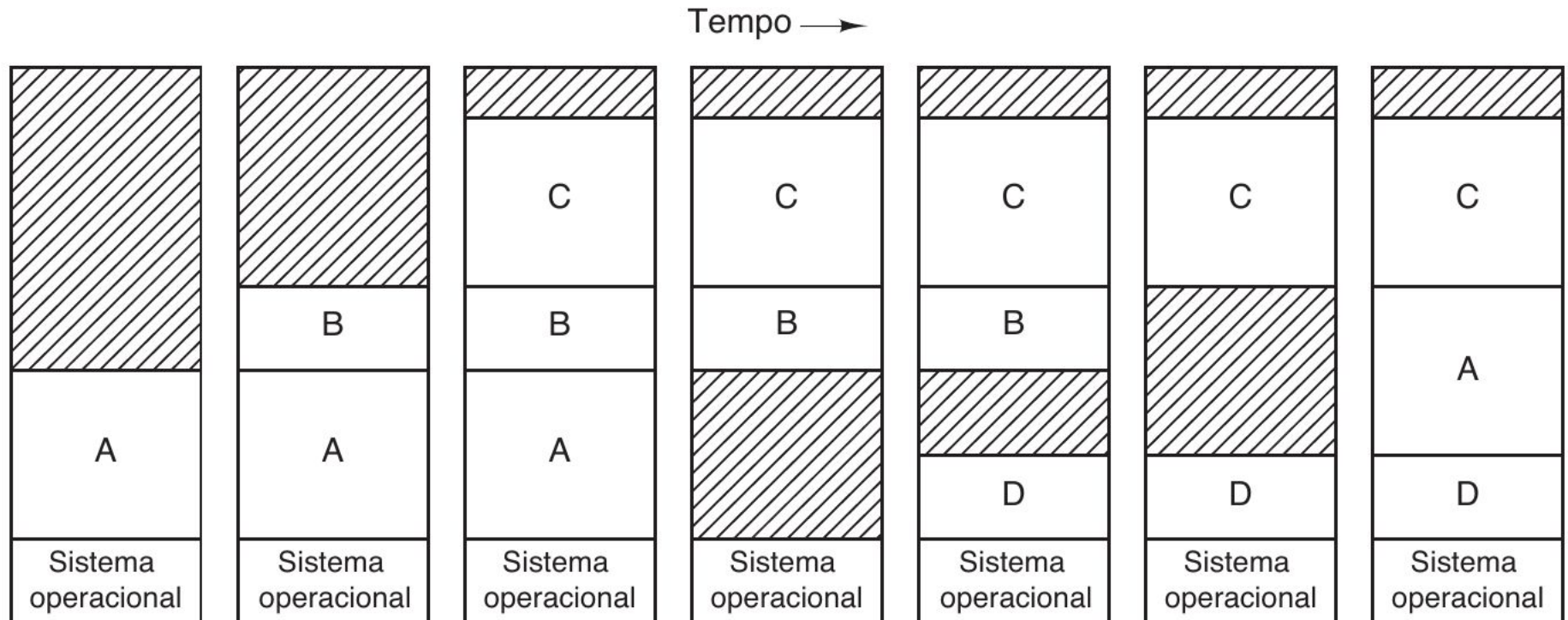
- O que é
 - Chaveamento de processos entre memória física e o disco
 - **Swap-out** - Da memória para o disco para uma área de swap
 - **Swap-in** - Do disco para a memória

Espaços de Endereçamento

- **Memória Virtual**

- Permitir que processos consigam executar, mesmo estando apenas parcialmente na memória principal
 - O usuário vê um espaço de endereçamento virtual, que pode ser igual, maior ou menor que a memória física

Espaços de Endereçamento



Espaços de Endereçamento

- Independente da estratégia, deve-se levar em consideração a quantidade de memória alocada por processo
 - Fixa para o processo (simples e “sem dor de cabeça”)
 - Crescimento dinâmico (realidade)
 - Processo quer mais memória que invadiria espaço de outro processo

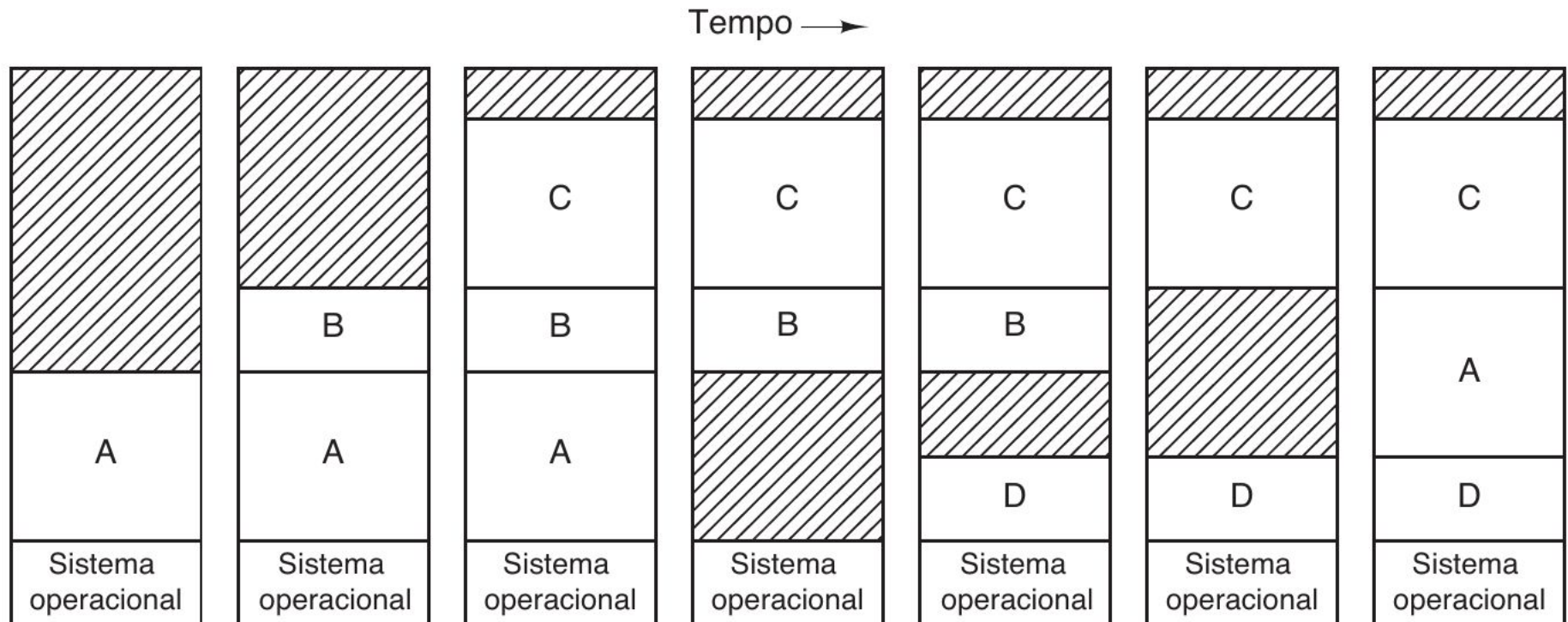
Tipos de Memória Particionada (1/2)

- **Partições Fixas** (ou alocação estática)
 - Tamanho e número da partição são fixos
 - Tendem a desperdiçar memória
 - Mais simples de implementar

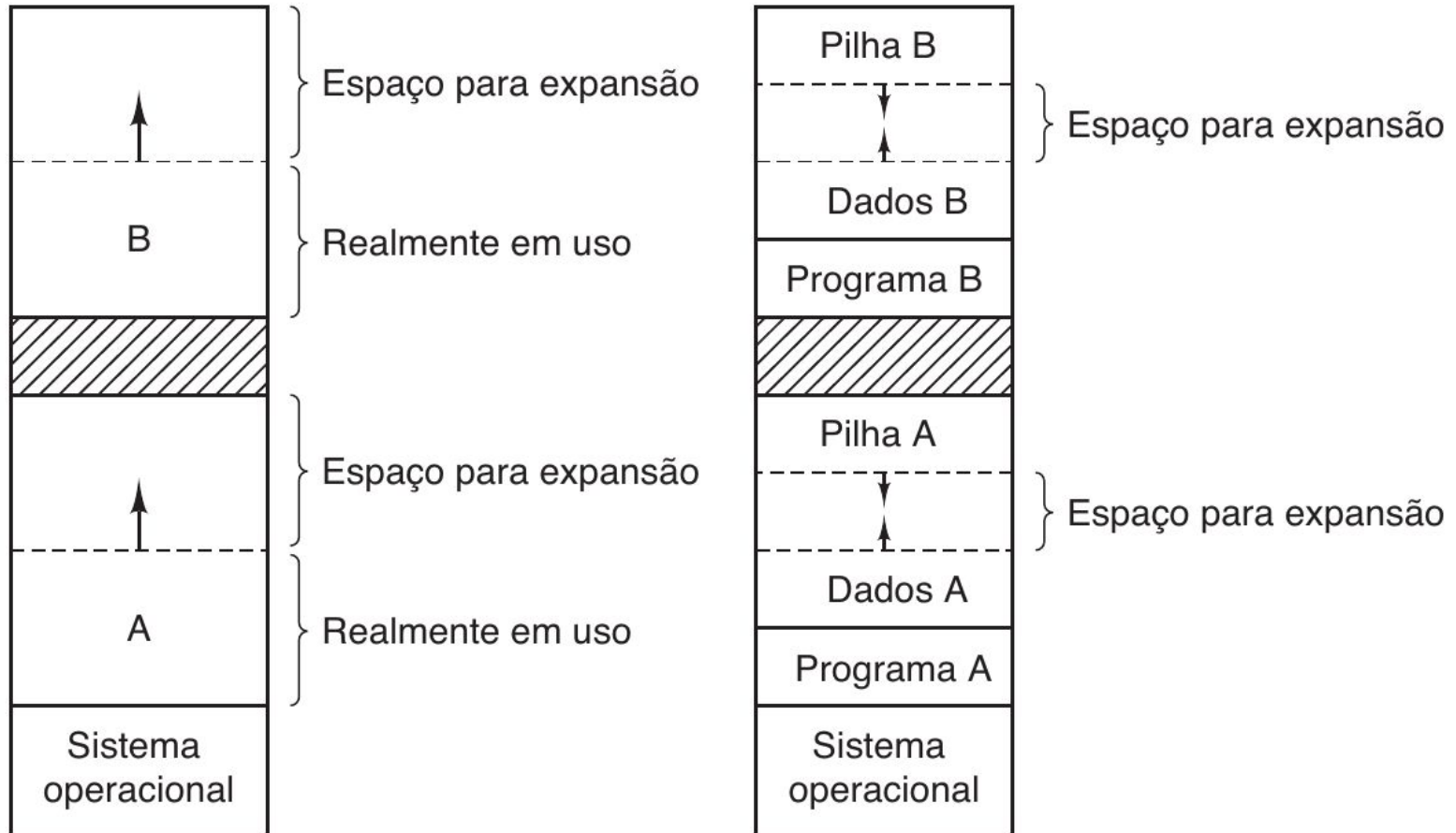
Tipos de Memória Particionada (2/2)

- **Partições Variáveis** (ou alocação dinâmica)
 - Tamanho e número de partições variam
 - Otimiza a utilização de memória
 - Complica em termos de alocação e liberação
 - Partições são alocadas dinamicamente
 - Pode causar fragmentação interna e externa

Espaços de Endereçamento



Espaços de Endereçamento



Gerenciamento de Memória Livre

Gerenciamento de Memória Livre

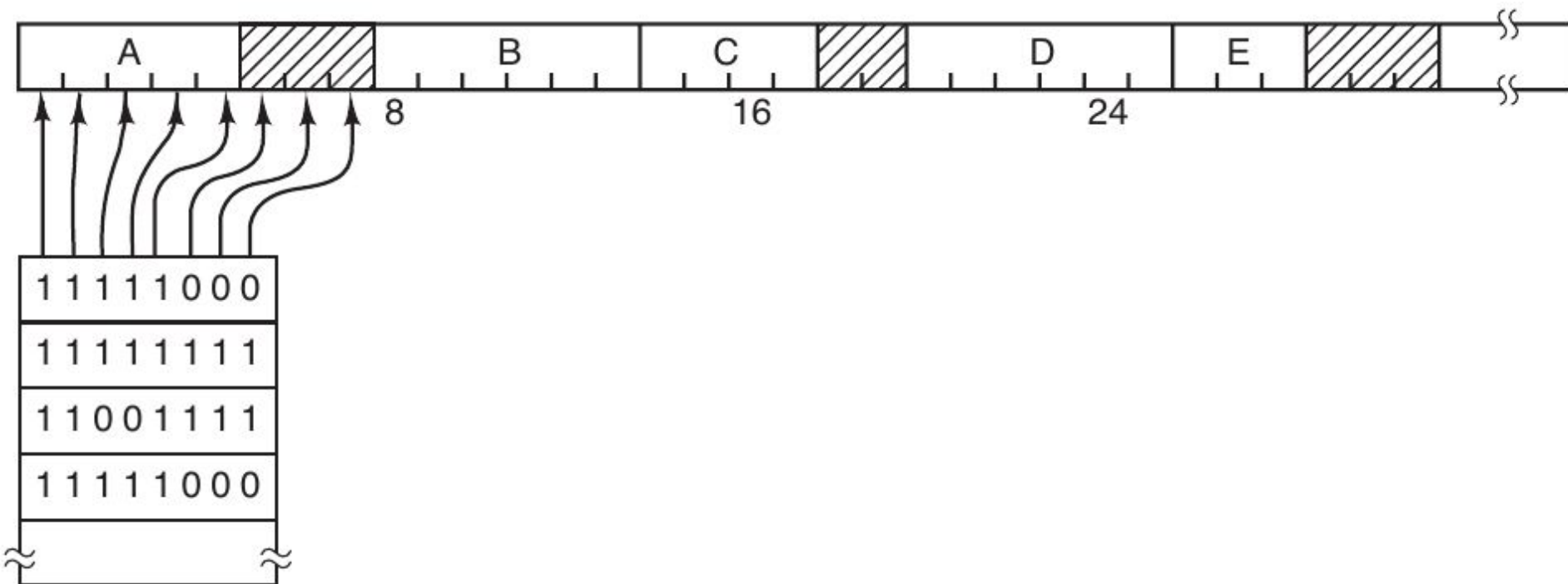
- Quando a memória é designada dinamicamente, esse é o caso onde o Sistema Operacional deve gerenciar
- Dessa forma, deve-se rastrear a memória em uso
- Há duas formas principais:
 1. **Gerenciamento por mapa de bits**
 2. **Gerenciamento por com listas encadeadas**

Gerenciamento de Memória Livre

- **Gerenciamento com Mapa de Bits (1/3)**
 - Memória é dividida em unidades de alocação pequenas
 - Para cada unidade, há um bit no mapa
 - 0 se estiver livre
 - 1 se estiver ocupado

Gerenciamento de Memória Livre

- Gerenciamento com Mapa de Bits



Gerenciamento de Memória Livre

- **Gerenciamento com Mapa de Bits (2/3)**
 - Tamanho da unidade de alocação é importante
 - O mapa cresce de acordo com esse tamanho
 - Se a unidade de alocação for grande, é onde ocorre o desperdício de memória

Gerenciamento de Memória Livre

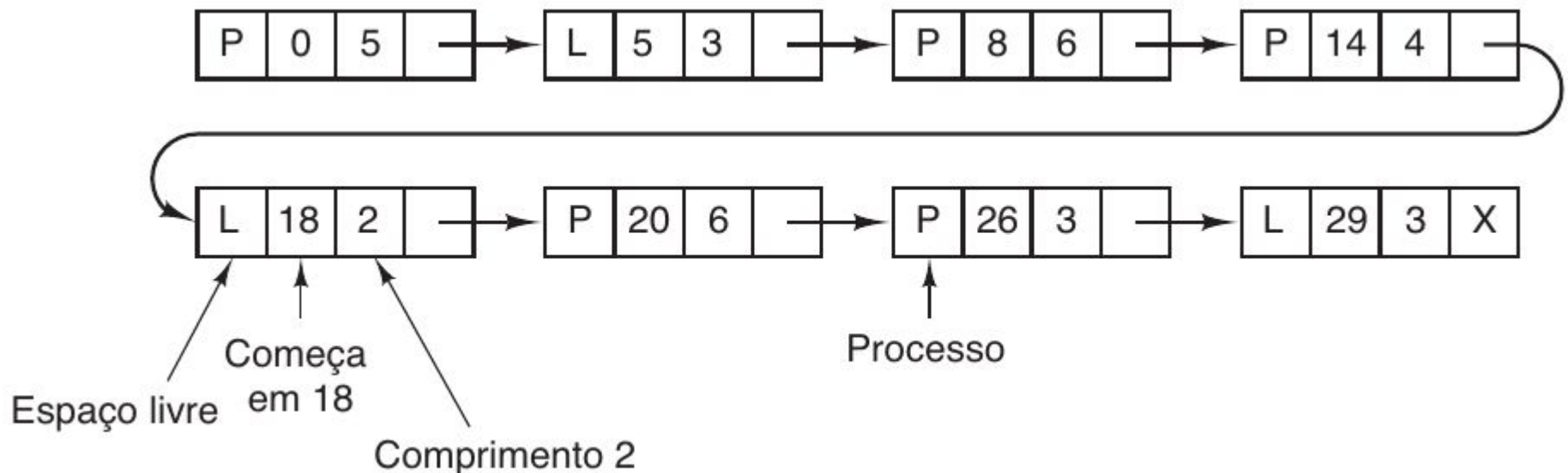
- **Gerenciamento com Mapa de Bits (3/3)**
 - Maneira simples de controlar
 - A complexidade vem de achar memória
 - Processo quer k unidades
 - Gerenciador precisa percorrer caçando por k
 - Operação lenta

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada (1/3)**
 - Gerencia uma lista encadeada de tamanho variável
 - Nós podem ser espaços livres ou um segmento alocado
 - O nó indica o tamanho do espaço utilizado

Gerenciamento de Memória Livre

- Gerenciamento por Lista Encadeada

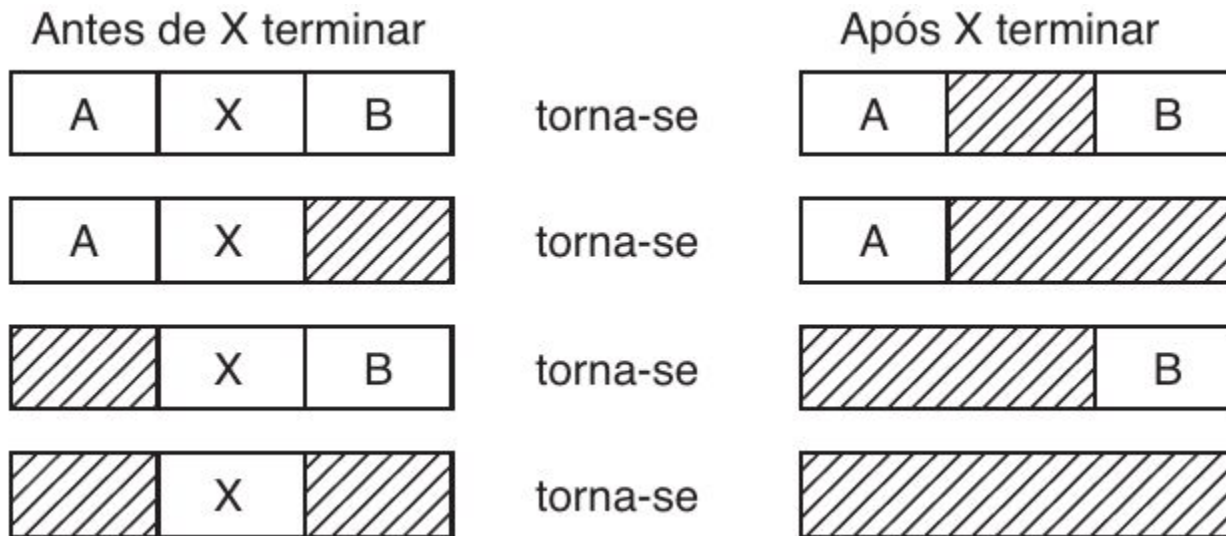


Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada (2/3)**
 - A lista é mantida ordenada
 - Atualização da lista é simples de realizar
 - Quando um processo termina ou atualiza, ele tem dois vizinhos (pode ser processo ou área livre)
 - Quatro combinações possíveis de atualização

Gerenciamento de Memória Livre

- Gerenciamento por Lista Encadeada



Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada (3/3)**
 - Apesar da figura, a lista é duplamente encadeada
 - Tem que mexer no nó anterior e posterior
 - No caso da lista, vários algoritmos diferentes podem ser utilizados para alocar memória para um processo

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **First Fit**
 - **Next Fix**
 - **Best Fit**
 - **Worst Fit**
 - **Quick Fit**

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **First Fit**
 - Gerenciador (sempre) sabe quanto de memória quer
 - Para todos os algoritmos
 - Percorre a lista até encontrar o primeiro espaço grande o suficiente (menor busca)

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Next Fit**
 - Acha a primeira ocorrência que cabe, grava e disponibiliza a encontrada para utilizar
 - Da próxima vez, começa a buscar a partir da gravada

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Next Fit**
 - Acha a primeira ocorrência que cabe, grava e disponibiliza a encontrada para utilizar
 - Da próxima vez, começa a buscar a partir da gravada
 - Parece bom, mas é pior que o First Fit (WTF né)

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Best Fit**
 - Varre toda a lista e devolve a menor possível
 - Mais lento por sempre varrer toda a lista

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Best Fit**
 - Varre toda a lista e devolve a menor possível
 - Mais lento por sempre varrer toda a lista
 - Parece bom, mas também é ruim, pois acaba fragmentando muito (WTF WTF, mas é verdade)

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Worst Fit**
 - Sempre escolhe o maior espaço, para fragmentar
 - Tenta resolver o problema do Best Fit
 - Memória que sobra é grande o suficiente para ser usável (boa a ideia, mas na prática também é ruim)

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Quick Fit (1/2)**
 - Mantém separado uma lista de espaços livres
 - Quando um processo quer memória, a lista de espaços liberados/livres é consultada
 - Pode manter listas de tamanhos padrões

Gerenciamento de Memória Livre

- **Gerenciamento por Lista Encadeada**
 - **Quick Fit (2/2)**
 - Encontrar espaço livre é bem mais rápido
 - Desvantagem ocorre na liberação de memória
 - Descobrir a vizinhança e fundir memória é caro
 - Sem fusão, a fragmentação tende a dominar

Gerenciamento de Memória Livre no Espaço de Usuário

Gerenciamento de Memória no Espaço de Usuário

- Da mesma maneira que o Sistema Operacional tem a necessidade de gerenciar blocos de memória alocadas a processos, processos também fazem gerenciamento de memória para tentar sempre evitar solicitar algo ao SO
 - Objetivo de reduzir *overhead* e utilizar aquilo que se já tem disponível

Gerenciamento de Memória no Espaço de Usuário

- Quando um processo é criado, ele recebe uma área de uso chamada de Heap, cujo o tamanho pode ser ajustado em tempo de execução
 - Esse funcionamento e ajuste dinâmico é feito por meio de bibliotecas do sistema
 - No caso do Linux padrão, a LibC

Gerenciamento de Memória no Espaço de Usuário

- **LibC (1/2)**
 - Provê funções de alocação (malloc) e liberação (free) de memória
 - Blocos liberados de memória (que com certeza é algo que o SO já reservou para o processo), vão para a área de gerenciamento da biblioteca ao invés de retornarem

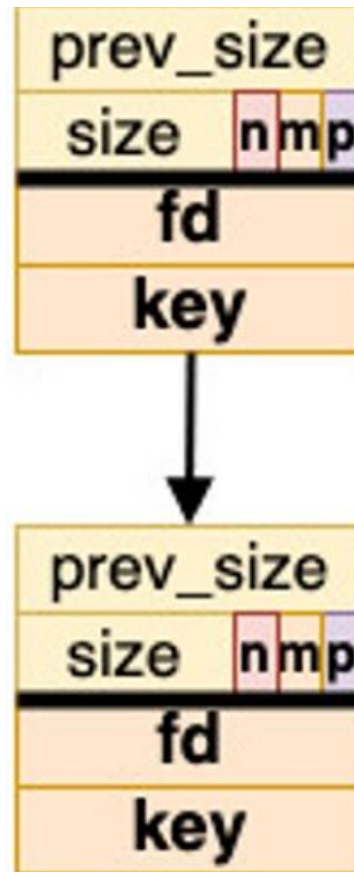
Gerenciamento de Memória no Espaço de Usuário

- **LibC (2/2)**
 - Existem diversas implementações de gerenciadores
 - dlmalloc (Doug Lea's malloc) - Mais famosa e mais genérica
 - ptmalloc2 (pthreads malloc) - Extensão da DLmalloc com threads
 - jemalloc - Desenvolvido para FreeBSD

Gerenciamento de Memória no Espaço de Usuário

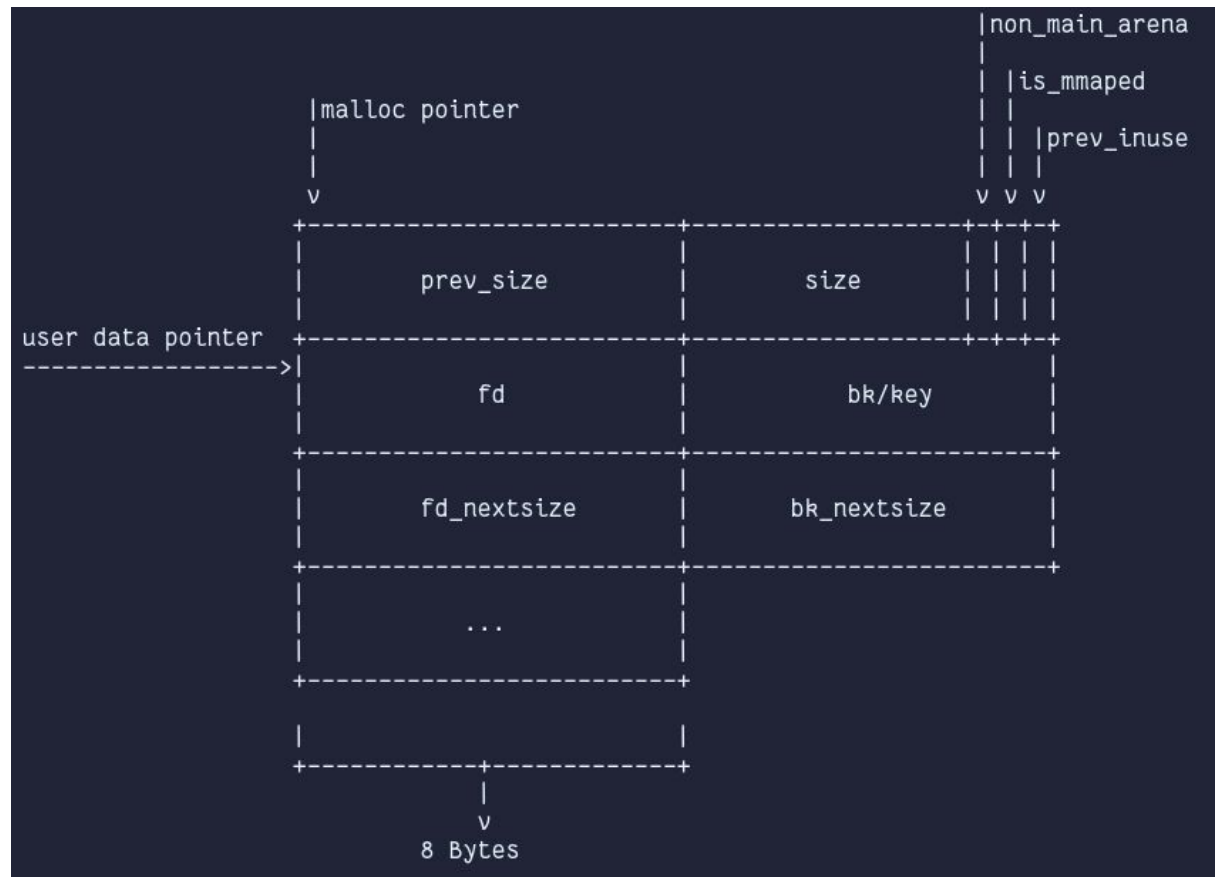
- **ptmalloc** (e por consequência, dlmalloc)
 - Porções de memórias liberadas são chamadas de bins
 - Memória liberada pelo programa vai para o gerenciador de memória
 - A própria área liberada é utilizada para guardar metadados de organização da memória liberada

Gerenciamento de Memória no Espaço de Usuário



Gerenciamento de Memória no Espaço de Usuário

- Chunk



Gerenciamento de Memória no Espaço de Usuário

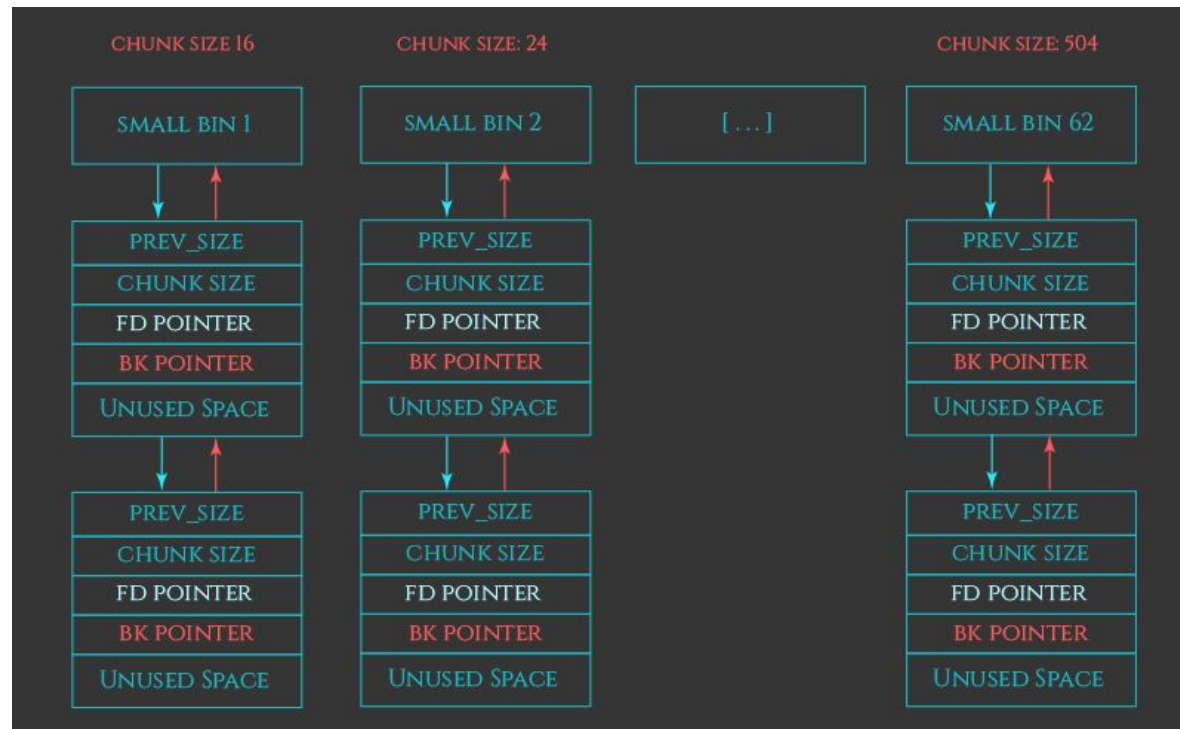
- **ptmalloc** (e por consequência, dlmalloc)
 - O gerenciador possui áreas de reciclagem de memória liberada
 - Múltiplas listas encadeadas chamadas de bins
 - 5 tipos de bins
 - 62 small bins
 - 63 large bins
 - 1 unsorted bin
 - 10 fast bins (otimização “recente”)
 - 64 tcache bins por thread (otimização “recente”)

Gerenciamento de Memória no Espaço de Usuário

- **ptmalloc** (e por consequência, dlmalloc)
 - small bins
 - Listas de até 512 bytes
 - Tamanho fixo
 - Rápidos

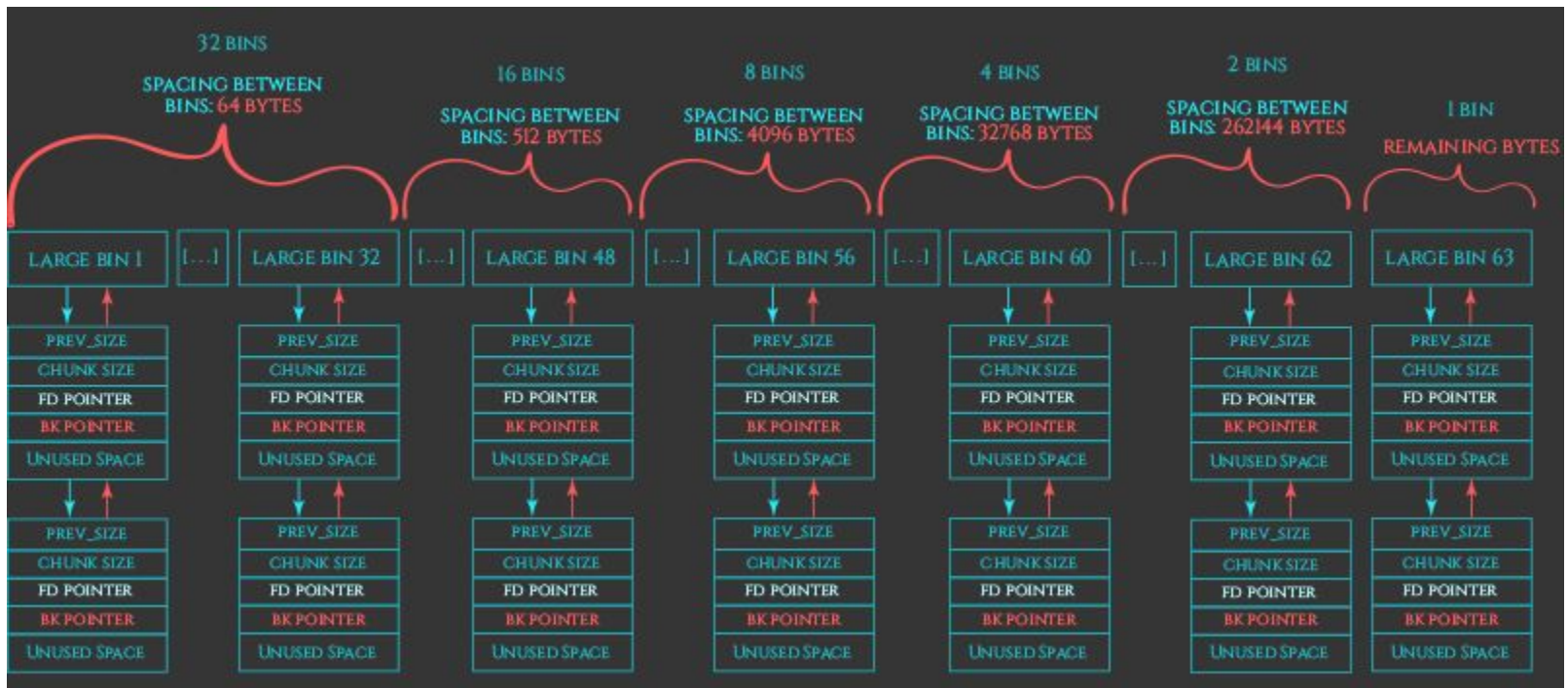
Gerenciamento de Memória no Espaço de Usuário

- **ptmalloc** (e por consequência, dlmalloc)



Gerenciamento de Memória no Espaço de Usuário

- ptmalloc (e por consequência, dlmalloc)



Conclusão